

Chapter 6

I/O Management

I/O (Input/Output) Management in an operating system is responsible for the smooth, efficient, and reliable interaction between the computer system and peripheral devices. It ensures that data can be transferred between memory, CPU, and devices without requiring the programmer to handle hardware-level complexities.

6.1. I/O Hardware

I/O Devices

I/O (Input/Output) devices are the hardware components used to **interact with a computer system**, either by supplying input, producing output, storing data, or enabling communication with other systems.

Variation in I/O Devices

I/O devices vary widely in the following aspects:

1. Speed

- Devices operate at different speeds based on their mechanical or electronic nature.
 - **Slow:** Printers (few pages per minute)
 - **Fast:** SSDs (hundreds of MB/s)
 - **Very Fast:** NVMe drives, RAM (GB/s range)

2. Data Format

- Devices transmit or receive data in various formats.
 - **Characters:** Keyboard (sends one character at a time)
 - **Blocks:** HDD, SSD (read/write in blocks like 512B, 4KB)
 - **Packets:** Network cards (send/receive IP packets)

3. Operations Supported

- Devices are designed for different I/O functions:
 - **Input:** Captures data from the external world.
 - **Output:** Sends processed data to users or other systems.
 - **Storage:** Saves data persistently.
 - **Communication:** Transfers data between systems.

I/O Devices Summary

Type	Description	Examples
Input Devices	Provide data to the computer	Keyboard, Mouse, Scanner, Microphone
Output Devices	Display or convey results to the user	Monitor, Printer, Speaker
Storage Devices	Retain data persistently	HDD, SSD, USB Drive, Optical Disks
Communication Devices	Enable data exchange with other systems	Network Interface Card (NIC), Modem

I/O Hardware Components

I/O hardware consists of **physical components** that allow communication between a computer system and external devices (peripherals). These components work together to **transfer data**, handle **control signals**, and ensure **synchronization** between the processor and devices.

i. I/O Devices (Peripherals)

- External or internal devices that perform input, output, storage, or communication.
 - **Input:** Keyboard, Mouse, Camera
 - **Output:** Monitor, Printer, Speaker
 - **Storage:** SSD, HDD
 - **Communication:** Network Interface Card (NIC), Modem

ii. Device Controller

- A hardware interface (usually a chip or circuit board) that connects the I/O device to the system bus and manages communication between the device and CPU.
 - Disk controller, USB controller, Display adapter

iii. I/O Ports

- The interface between the CPU and the I/O controller; a set of **memory-mapped** or **dedicated registers** used to send and receive data or commands.
 - Ports associated with USB, serial (COM), or parallel (LPT) interfaces.

- **Example**
 - **Control Port:** For commands like "start" or "stop".
 - **Status Port:** Returns information like "busy" or "ready".
 - **Data Port:** Used to transfer actual data.

iv. System Bus

- A communication pathway that connects the CPU, memory, and I/O devices.
 - PCIe, USB, SATA, and older buses like ISA or PCI.
- Common Types
 - **Data Bus:** Transfers actual data.
 - **Address Bus:** Identifies the device or memory location.
 - **Control Bus:** Carries signals like read/write or interrupt requests.

v. Interrupt Mechanism

- A hardware signal sent by an I/O device to request CPU attention.
- Keyboards interrupt when a key is pressed; network interrupt on receiving a packet.
- **Purpose:**
 - Notifies CPU that an event (e.g., data ready, transfer complete) has occurred.
 - Allows asynchronous communication between CPU and devices.

vi. Direct Memory Access (DMA) Controller

Will discuss it in communication methods

Data Transfer Methods\ Communication methods

Data Transfer Method	How it Works	CPU Involvement	Typical Devices / Operations in a Laptop
Programmed I/O (Polling)	CPU repeatedly checks device status and transfers data itself	High	• Battery status check • Simple hardware checks during boot (BIOS/UEFI) • Low-level controller checks
Interrupt-Driven I/O	Device sends interrupt signal when it needs attention	Medium	• Keyboard input • Mouse movement • Touchpad clicks • Wi-Fi notifications • Printer signals

Direct Memory Access (DMA)	DMA controller transfers data directly between device and RAM	Low	• SSD/HDD data transfer • GPU/graphics processing • Audio/video streaming • USB file transfer
-----------------------------------	---	-----	--

a. Programmed I/O (Polling)

In programmed I/O, the CPU actively monitors the status of an I/O device by repeatedly reading a status register to check whether the device is ready to send or receive data. This constant checking is known as *polling*. Only when the CPU detects that the device is ready does it proceed to transfer data by reading from or writing to the device's data register.

Because the CPU is busy looping and checking the device status, it cannot perform other tasks efficiently during this period.

Device Type

- Devices with low data rates or infrequent I/O operations.
- Simple hardware devices without interrupt capabilities.
- Embedded or specialized systems with simple timing requirements.

Example

- **Keyboards:** When a key is pressed, the CPU polls the keyboard controller to check if a keycode is ready. Since keystrokes are relatively slow, polling is manageable.
- **Simple sensors:** Some embedded systems read sensor values at fixed intervals by polling the sensor's status register.
- **Early microcontrollers:** Many microcontrollers with limited hardware resources rely on polling to manage I/O devices.

b. Interrupt-Driven I/O

In interrupt-driven I/O, the CPU issues a command to the device and then continues with other processing tasks. When the device is ready to transfer data or has completed an operation, it sends an **interrupt signal** to the CPU. This signal temporarily suspends the CPU's current work, and the CPU executes an **Interrupt Service Routine (ISR)** to handle the data transfer or device status.

After the ISR completes, the CPU resumes its previous task. This mechanism allows the CPU to perform other useful work rather than waiting actively.

6.2 IO Software

2.1. Goals of I/O Software

i. Device Independence

- The I/O software should allow programs to **read and write data from different devices** (e.g., floppy disk, hard disk, CD-ROM) without requiring changes to the application code.
- **Goal:** Allow the same code to work with different types of devices without modification.
- **Explanation:** Programs should not need to know whether they're reading from a floppy disk, hard drive, USB, or CD-ROM. The I/O software abstracts away the hardware-specific details.
- **Example:**
 - A media player application can **open and play a music file** from a CD-ROM, a USB drive, or the internal SSD without changing its source code.

ii. Uniform Naming

- Devices and files should be named in a way that is **independent of the actual hardware device**.
- **Goal:** Use a consistent naming scheme for accessing files and devices, regardless of hardware.
- **Explanation:** Naming should not depend on the device's specific characteristics. All devices and files should be accessed through a consistent namespace (like a file path).
- **Example:**
 - In UNIX/Linux:
 - `/dev/sda` refers to the first SATA hard disk.
 - `/dev/ttyUSB0` refers to the first USB serial device.
 - `/home/user/file.txt` is a file on a disk but is accessed the same way as `/dev/sda` if needed.
 - The command `cat /dev/sda` will try to read raw data from a device just like it reads from a regular file.

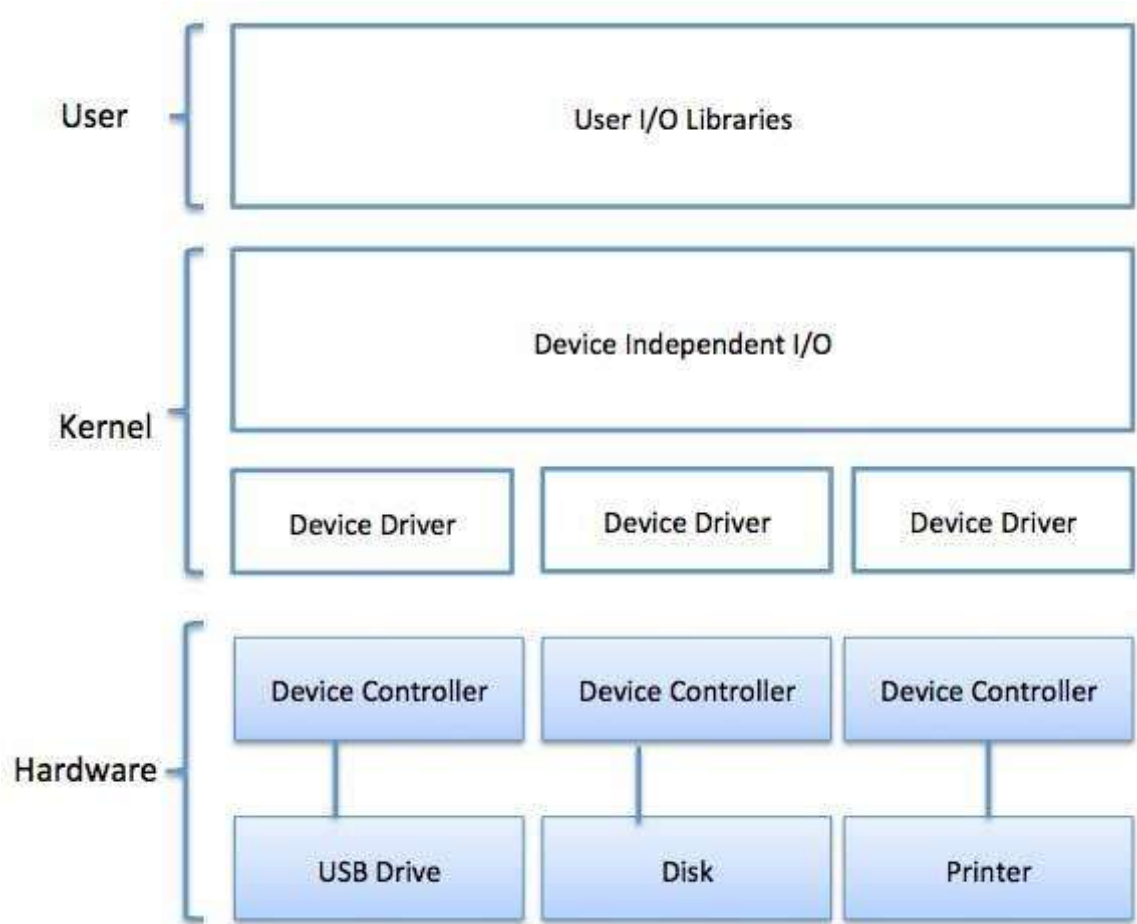
iii. Error Handling Done as Close to Hardware as Possible

- Errors should be handled **as close to the hardware layer as possible** to minimize propagation and complexity.
- **Goal:** Handle recoverable errors at the lowest level, only escalating when necessary.
- **Explanation:** Transient errors like read/write failures or signal interruptions are often recoverable by the hardware or driver. They shouldn't interrupt the application unless absolutely necessary.
- **Example:**
 - A disk read error might be retried **automatically** by the disk controller or driver up to 3 times before reporting failure.
 - If a CD has a scratch, the **driver** may use **error-correcting codes (ECC)** or retry strategies without informing the user.

iv. Synchronous Read/Write at Application Level

- Most I/O devices operate **asynchronously** — meaning they perform data transfers independently of the CPU. However, for ease of programming, the I/O software provides **synchronous read/write operations** to applications.
- **Goal:** Hide the complexity of asynchronous hardware operations from the application.
- **Explanation:** Although most devices operate asynchronously (they take time to complete operations), the software interface often provides **blocking** (synchronous) calls that wait until the operation completes, simplifying programming.
- **Example: Writing to a USB stick**
- When an application performs a write() operation to a USB drive, the operating system:
 1. Buffers the data,
 2. Schedules the write in the background,
 3. Waits for hardware to confirm that the operation is completed.
- From the application's perspective, it **waits** (blocks) until the OS says the data is safely written, even if it was actually handled asynchronously behind the scenes.

2.2. I/O Software Layers



1. User Layer

- **Component: User I/O Libraries**
- **Role:**
 - Applications interact with I/O through library functions such as `printf()`, `scanf()`, `fread()`, `fwrite()` in C or equivalent APIs in other languages.
 - These library calls are translated into **system calls** (`read()`, `write()`, `open()`, `close()`) which the kernel can understand.
- **Example:**
 - When a program uses `fopen()` to open a file on a USB drive, it doesn't worry about whether the storage is USB, HDD, or SSD.

2. Kernel Layer

This layer handles **core I/O processing** inside the OS.

(a) Device-Independent I/O

- **Role:**
 - Provides a **uniform interface** to all devices, hiding hardware-specific details.
 - Manages **buffering, caching, and error handling**.
 - Handles **device naming and access control** (e.g., /dev/usb, /dev/sda).
- **Example:**
 - A read request from a disk or USB will go through the same generic file system interface.

(b) Device Drivers

- **Role:**
 - Contain **device-specific logic** to communicate with hardware.
 - Convert generic commands from the device-independent layer into **low-level instructions** understood by the hardware controller.
- **Example:**
 - USB driver sending the correct packet format to a USB controller.
 - Printer driver formatting data for the printer hardware.

3. Hardware Layer

This is where the actual I/O devices exist.

(a) Device Controllers

- **Role:**
 - Act as intermediaries between device drivers and the physical hardware.
 - Receive instructions from the driver and translate them into **electrical signals** to operate the device.
 - Provide **status registers, data buffers**, and sometimes support for **DMA**.
- **Example:**
 - USB controller managing data transfer between USB drive and memory.
 - Disk controller handling read/write commands.

(b) I/O Devices

- **Examples:**
 - **USB Drive:** Storage for files.
 - **Disk:** HDD or SSD used as primary storage.
 - **Printer:** Produces hard copies of documents.

Common Example: Flow of an I/O Operation (Example: Printing a File)

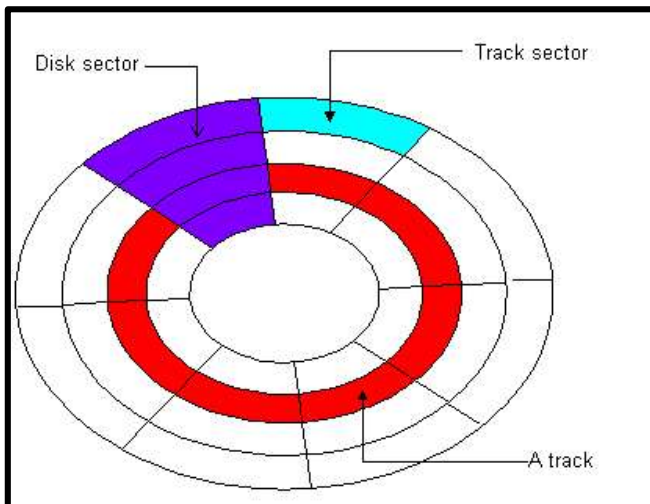
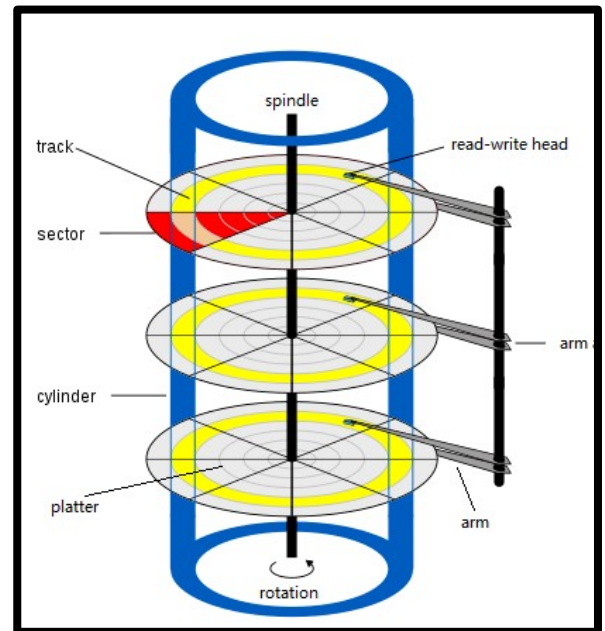
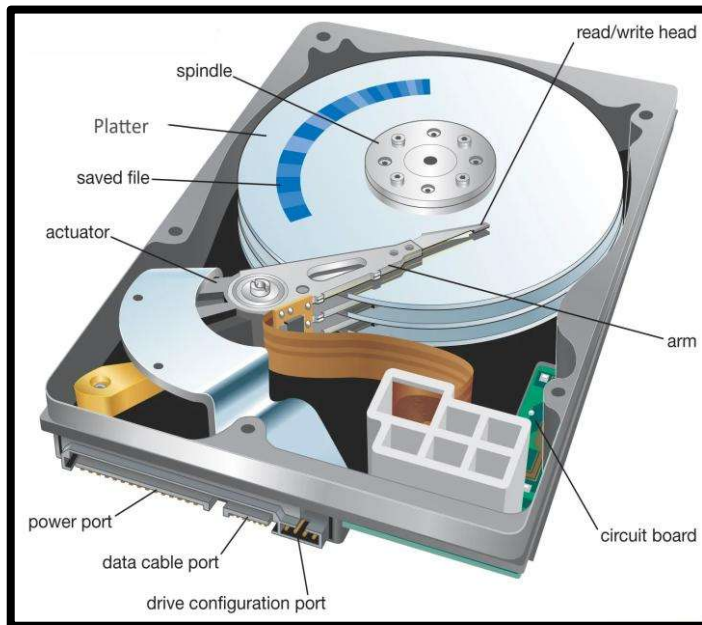
1. **User Program (User I/O Library):** Calls print command.
2. **Device-Independent I/O (Kernel):** Passes request in a uniform way to the printer driver.
3. **Printer Driver:** Translates print command into printer-specific control codes.
4. **Printer Controller (Hardware):** Sends the control signals to the physical printer.
5. **Printer:** Produces the physical output.

6.3. Disk

3.1. Hardware

A disk (hard disk drive – HDD) is a non-volatile storage device that stores data magnetically on rotating platters.

Main Components:



- **Platters** – Circular disks coated with magnetic material.
- **Spindle** – Holds platters and rotates them at constant speed (e.g., 5400, 7200, 15000 RPM).
- **Read/Write Heads** – Electromagnetic heads that read/write data without touching the platter surface.
- **Actuator Arm** – Moves the head across the disk.
- **Controller** – Embedded electronics that manage data transfer, caching, and communication with the OS.

- **Performance Factors:**
 - **Seek time** (time to move head to correct track).
 - **Rotational latency** (time for correct sector to rotate under head).
 - **Data transfer rate** (how fast data is read/written).
 - **Cache size** (onboard memory for buffering).
- **Comparison with SSD:**
 - HDD – mechanical movement, cheaper per GB, higher latency.
 - SSD – no moving parts, faster, more expensive per GB.

3.2. Formatting

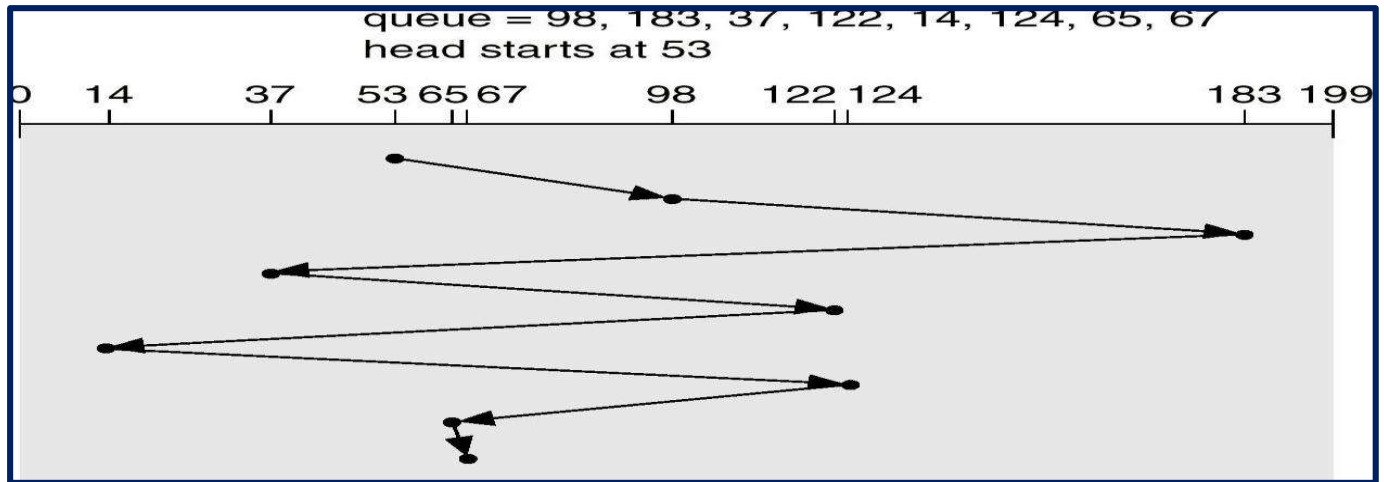
- **Low-Level Formatting (Physical Formatting):**
 - Divides disk into **tracks** and **sectors**.
 - Usually done at the factory.
- **Partitioning:**
 - Divides the disk into partitions, each treated as a separate logical disk.
 - Partition table formats:
 - ✓ **MBR (Master Boot Record)** – older, supports up to 2TB.
 - ✓ **GPT (GUID Partition Table)** – newer, supports disks >2TB and more partitions.
- **High-Level Formatting:**
 - Creates a file system on the partition (e.g., NTFS, ext4, FAT32).
 - Initializes boot sectors, file allocation tables, superblocks.

3.3. Arm Scheduling

- It decides the order in which requests are serviced.
- **Algorithms:**
 - **FCFS (First Come First Serve)** – Serves requests in arrival order.
 - **SSTF (Shortest Seek Time First)** – Chooses request closest to current head position.
 - **SCAN (Elevator Algorithm)** – Moves arm in one direction serving requests, then reverses.
 - **C-SCAN (Circular SCAN)** – Like SCAN, but after reaching one end, jumps back to start without servicing requests in reverse direction.
 - **LOOK / C-LOOK** – Like SCAN/C-SCAN, but only goes as far as the furthest request in each direction.
- **Example:**
 - Head starts at track 53, requests: 98, 183, 37, 122, 14, 124, 65, 67.
 - SSTF sequence: 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183.
- **Performance Metric:** Average seek time, total head movement.

1. FCFS

- In FCFS the requests are addressed in the order they arrive in the disk queue. For example, consider a disk with 200 tracks.

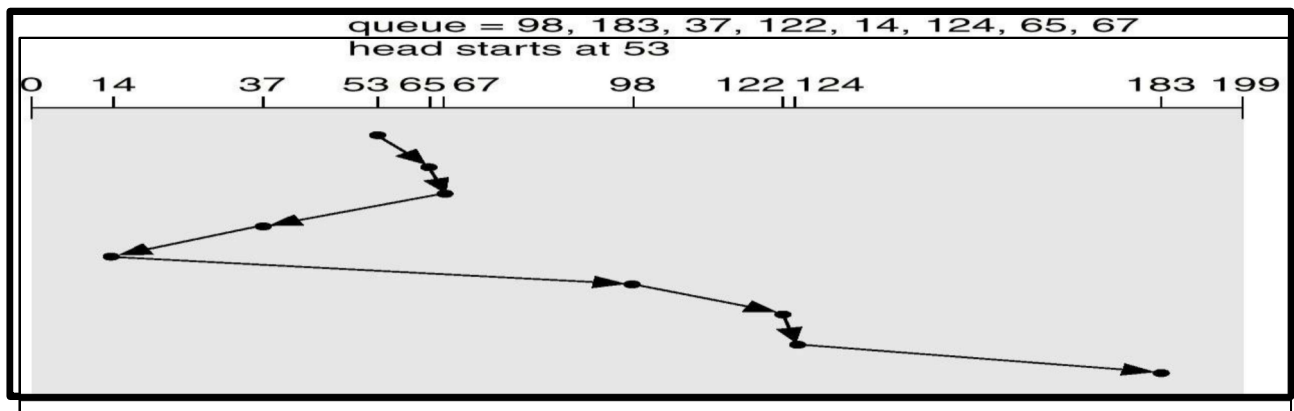


- Total head/track movement**

$$= (98-53) + (183-98) + (183-37) + (122-37) + (122-14) + (124-14) + (124-65) + (67-65) = 640 \text{ tracks}$$

2. SSTF(shortest seek time first)

- In SSTF request having the shortest seek time are executed first.

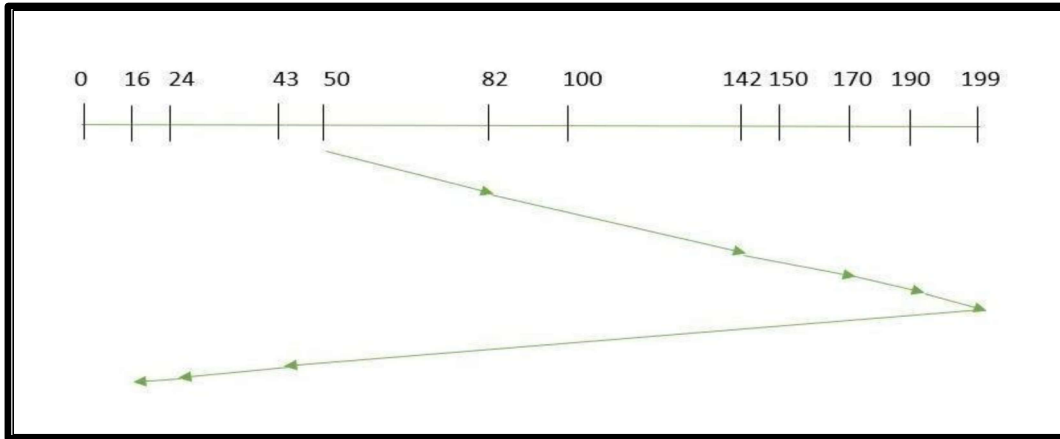


- Total head/track movement**

$$= (65-53) + (67-65) + (67-37) + (37-14) + (98-14) + (122-98) + (124-122) + (183-124) = 236 \text{ tracks}$$

3. SCAN

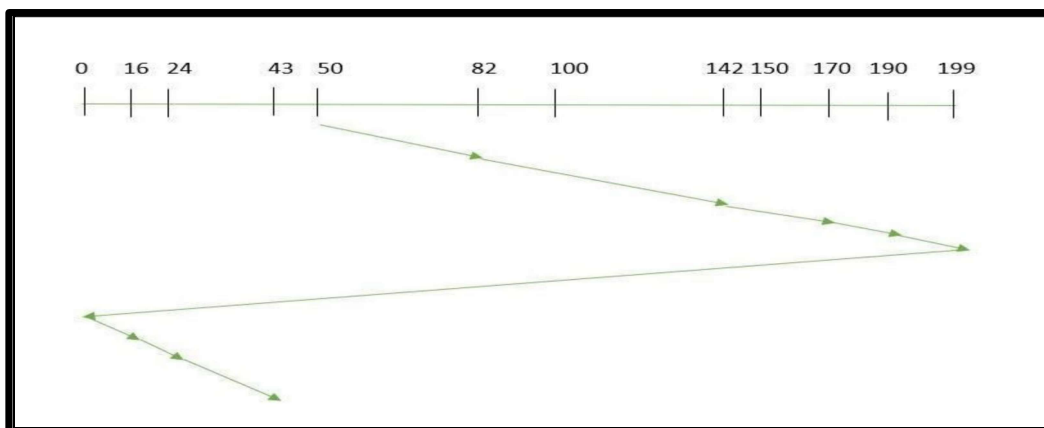
- The disk arm moves in a particular direction till the end, satisfying all the requests coming in its path, and then it turns back and moves in the reverse direction satisfying requests coming in its path. Also called Elevator Algorithm because, It works in the way an elevator works
- Suppose the requests to be addressed are 82,170,43,140,24,16,190. And the Read/Write arm is at 50, and the disk arm should move “towards the larger value”.



- **Total head/track movement**
 $= (199-50) + (199-16) = 332$ tracks

4. C-SCAN

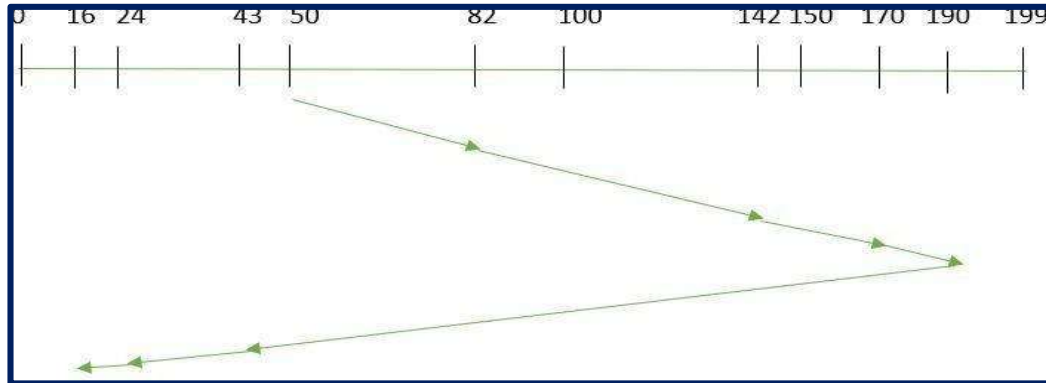
- The head moves from one end of the disk to the other servicing requests as it goes. When it reaches the other end, it immediately returns to the beginning of the disk, without servicing any requests on the return trip.
- Suppose the requests to be addressed are 82,170,43,140,24,16,190. And the Read/Write arm is at 50, and the disk arm should move “towards the larger value”.



- **Total head/track movement**
 $= (199-50) + (199-0) + (43-0) = 391$ tracks

5. LOOK

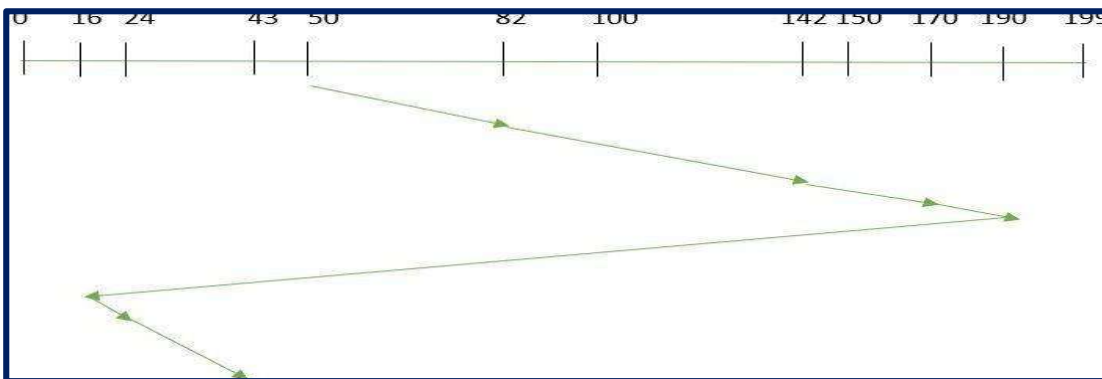
- It is similar to the SCAN, except that the disk arm goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.
- Suppose the requests to be addressed are-82, 170, 43, 140, 24, 16, and 190. And the Read/Write arm is at 50, and the disk arm should move “towards the larger value”.



- Total head/track movement**
 $= (190-50) + (190-16) = 314 \text{ tracks}$

6. C-LOOK

- Version of C-SCAN, where arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk.
- Suppose the requests to be addressed are-82, 170, 43, 140, 24, 16 and 190. And the Read/Write arm is at 50, and the disk arm should move “towards the larger value”.



- Total head/track movement**
 $= (190-50) + (190-16) + (43-16) = 341 \text{ tracks}$

6.4. Stable Storage

Stable storage **refers to storage that is designed to survive:**

- **Power failures**
- **System crashes**
- **Hardware failures**
- It ensures that once data is written, it remains correct and accessible even after unexpected interruptions.
- Implemented using **redundant hardware and software techniques** to protect data.

Key Idea: Data in stable storage is as safe and permanent as practically possible within the system's constraints.

4.4.1. How Stable Storage is achieved

Hardware Redundancy

1. Mirrored Disks (RAID 1)

- Data written to two or more disks.
- **Example:** A banking database server stores its transaction log on two SSDs in RAID 1.
- **Benefit:** If one SSD fails, the identical copy on the other drive keeps the database safe.

2. Error-Correcting Codes (ECC)

- Detect and correct small errors.
- **Example:** A workstation with ECC RAM automatically detects and corrects single-bit errors when reading OS kernel data, preventing crashes.
- **Benefit:** Avoids silent data corruption.

Software Techniques

3. Write-Ahead Logging (WAL)

- Changes are logged before being applied.
- **Example:** PostgreSQL database writes a transaction to its WAL file before updating tables.
- **Benefit:** If the system crashes, the database can replay the log to restore consistency.

4. Atomic Writes

- Ensures partial writes don't corrupt data.
- **Example:** When you save a Word document on Windows (NTFS) or a text file on Linux (ext4), the file system doesn't overwrite the file directly.
 - It first writes the update to a **journal entry**.
 - Then it applies the update to the file **in a single atomic step**.
- **Benefit:** If the computer loses power mid-save, the journal ensures the file is either:
 - **Completely updated**, or
 - **Left unchanged in its last consistent state**.
- **Outcome:** Prevents **"half-written" or corrupted files** after crashes.

5. Checksums

- Detect data corruption.
- **Example:** ZFS file system stores a checksum for each data block. When reading, it verifies the checksum to detect corruption.
- **Benefit:** Corrupted blocks are automatically repaired (if redundancy is available).

Generic Example 1

Imagine you are working on a document in a word processor on your PC.

- When you click **Save**, the program doesn't just write the file data directly to disk immediately.
- Instead, the OS and file system use **stable storage techniques** such as **journaling**:
 - The changes are first recorded in a **journal/log** stored on the disk.
 - This journal acts as a safety checkpoint.
- If the computer **suddenly loses power** or crashes during saving:
 - Upon reboot, the file system checks the journal.
 - It uses the journal to **complete or roll back** incomplete changes, preventing file corruption.
- As a result, your document remains intact and consistent, even if the power went out mid-save.

Summary: Stable storage mechanisms in everyday PCs protect your files from corruption caused by unexpected shutdowns by carefully managing how data is saved to disk.

Generic Example 2

Consider an online banking system where a customer transfers money from their savings account to their checking account.

- When the customer initiates the transfer, the database system first **writes a commit log entry** to **stable storage** indicating the transaction details (debit savings, credit checking).
- This log entry is **persisted on redundant disks** with error checking to ensure it won't be lost, even if the power fails immediately after.
- Only after the log is safely stored does the system actually update the account balances.
- If the system crashes mid-transaction, the bank's recovery process reads the stable storage logs and either:
 - **Completes the transaction** (if the log indicates it was committed), or
 - **Rolls back** any partial changes to maintain consistency.

Outcome:

Thanks to stable storage, the customer's money is never lost or duplicated, even if the system fails unexpectedly during the transaction.